

지사에서 파이썬 활용

세종지사 Power-UP day

2026. 6. 24

 국토정보교육원



강사

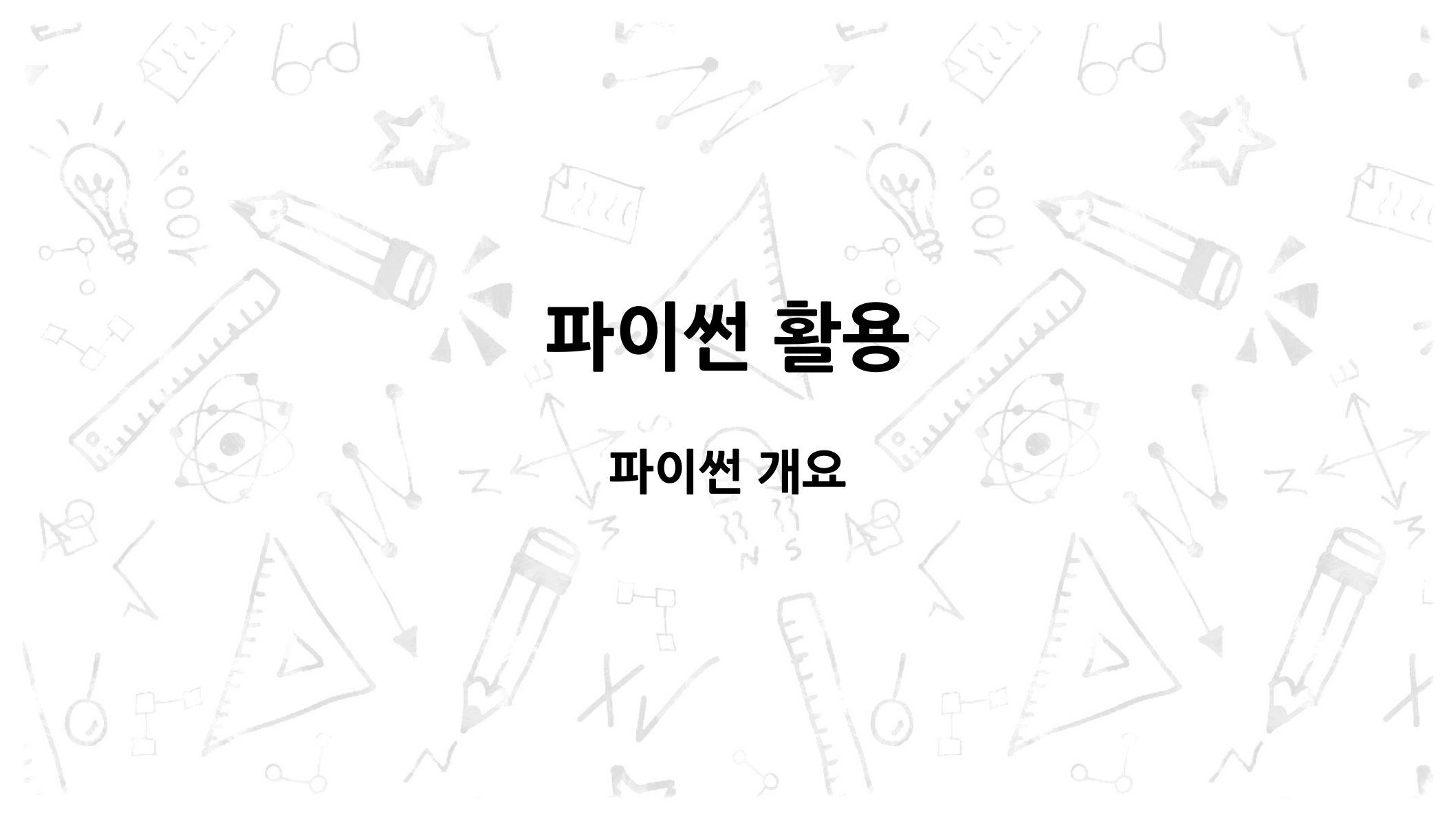
- Eugene Kim
- 국토정보교육원 인재개발실 전임강사
- blog: <https://profkim2000.github.io/blog>





Contents

1. 파이썬 개요
2. 프로그래밍 이야기
3. 파이썬 기초
4. AI로 파이썬 프로그램 개발하기
5. 측량과 공간정보에 사용하는 파이썬



파이썬 활용

파이썬 개요

python(파이썬)

- **원래 뜻**

- 비단뱀
- 고대 신화에 나오는 큰 뱀

- **프로그래밍 언어 이름에 python을 붙인 이유**

- python 제작자(귀도 반 로섬)이 좋아하는 코미디 Monty Python's Flying Circus에서 따온 이름. <출처: 위키백과>

python(파이썬)

- 로고



- 뱀 2마리가 얹혀 있는 형상

사람들이 파이썬에 열광하는 이유

▪ 쉽다

- 파이썬 자체가 쉽다기보다는 다른 프로그래밍 언어들에 비해서는 낫다
- 개발에 필요한 많은 기능들이 준비되어 있다
 - 예) PDF 파일을 읽어서 화면에 보여줄 필요가 있을 때 PDF 기능을 직접 개발하는 것이 아닌, 가져와서 쓰면 된다.
 - 다른 사람들이 만든 기능을 쉽게 가져와 쓰는 환경이 잘 갖춰져 있다.
- 초보 수준에서 지식이 얇아도 다른 언어에 비해 많은 일을 할 수 있다

사람들이 파이썬에 열광하는 이유

- 간결하다

다른 언어: C언어	python
<pre>int a = 10; int b = 20; int c = a + b; printf("c = %d", c);</pre>	<pre>a = 10 b = 20 c = a + b print("c = ", c)</pre>

사람들이 파이썬에 열광하는 이유

- 무수히 많은 참고 자료

- 사용자 수가 많으니 참고 자료도 많다
 - 책, blog, youtube 등
- 개발에 필요한 많은 기능들이 준비되어 있다
 - 예) PDF 파일을 읽어서 화면에 보여줄 필요가 있을 때 PDF 기능을 가져와서 쓰면 된다.
 - 다른 사람들이 만든 기능을 쉽게 가져와 쓰는 환경이 잘 갖춰져 있다.

사람들이 파이썬에 열광하는 이유

- **무수히 많은 라이브러리**

- 라이브러리(library)
 - 어떤 기능을 만들어 미리 놓은 것
 - 예) PDF 라이브러리: PDF 편집 프로그램 만들 때 사용
 - 예) 엑셀 라이브러리: 엑셀 편집 프로그램 만들 때 사용
 - 예) 이미지 라이브러리: 반지름이 4cm인 원을 그릴 때 사용
- 필요한 기능 웬만한 건 이미 라이브러리로 개발되어 있더라

사람들이 파이썬에 열광하는 이유

- AI, 빅데이터 분석용 라이브러리가 대거 공개되어 있다.
 - 파이썬 자체가 AI, 빅데이터 분석에 특화되어 있다기 보다는 언어 자체가 쉬우니 초보자들이 많이 사용하고 있고, 그래서 라이브러리를 파이썬 버전으로 많이 만들어 공개함.
 - 많은 수의 사용자들이 더 많은 라이브러리들을 만들어 공개하고 그에 따라 사용자가 더 늘어나는 선순환이 이루어지고 있는 상태

사람들이 파이썬에 열광하는 이유

- 설치와 사용이 쉽다

- 무료
- python 설치 프로그램 크기가 작아 금방 설치하고 바로 사용할 수 있다.
 - java는 설치 후 환경설정을 해야 한다.

python으로 할 수 있는 것들

- 웹사이트

- 내 컴퓨터가 웹서버가 된다.
- 내가 만든 웹사이트에, 내 친구가 웹브라우저로 접속해서, 파일을 업로드하거나 다운로드하게 개발할 수 있다.

python으로 할 수 있는 것들

▪ AI

- 많은 AI 라이브러리들이 python을 사용해 개발되어 AI를 하려면 python을 배워야 할 정도
 - 텐서플로
 - 사이킷-런
 - 케라스
 - 파이토치

python으로 할 수 있는 것들

▪ 데이터 분석

- 많은 데이터 분석용 라이브러리들이 개발, 공개되어 데이터 분석에 적합한 언어로 인식되고 있다.
 - Pandas
 - Numpy
 - Matplotlib

python으로 할 수 있는 것들

- IoT(Internet of Things, 사물 인터넷)

- IoT의 선두주자 라즈베리파이(raspberry pi)에서 python을 기본 언어처럼 사용
 - 라즈베리파이에 연결된 각종 센서들의 값을 python으로 분석하고 서버에 기록하는 등의 작업

python이 하기 어려운 일

- 실시간 작업

- real time이라 불리는 빠른 처리를 요하는 작업
- python은 C언어보다 처리속도가 많이 느리다.

인터프리터 언어, 대화형 언어

- 파이썬을 부르는 말들

- 인터프리터 언어
- 대화형 언어

python 코드

```
>>> a = 10  
>>> b = 20  
>>> a + b  
30
```

설명

a는 10이야.
b는 20이야.
a와 b를 더한 값을 보여줘
30

- 사람과 컴퓨터가 대화하듯 컴퓨터에게 정보를 알려주고 그 결과를 받는 방식

인터프리터 언어, 대화형 언어

▪ 인터프리터 언어가 아닌 언어

- C언어, java 등
- 프로그래밍 결과 .exe 파일이 생성된다.(python은 .exe가 생성 안됨)

c 코드

```
int a = 10;  
int b = 20;  
printf("%d", a + b);
```

설명

a는 10이야.
b는 20이야.
a와 b를 더한 값을 보여줘

prog1.exe

파일이 생성됨. exe 파일 실행 후 결과 볼 수 있음.

python 설치

- python 다운로드

- URL: <https://www.python.org>
- 설치할 python version: 3.13.2
- 설치 방법 안내:

https://profkim2000.github.io/blog/dev/python/install_python



python 맛보기

▪ python 실행

- 윈도우 키를 누른 후 "cmd" 입력 => 명령 프롬프트 실행
- 명령 프롬프트에서 "python" 실행

```
Microsoft Windows [Version 10.0.19045.4651]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\eugene>python
```

python 맛보기: 대화형 언어 경험하기

■ 덧셈

```
>>> 1+1
2
```

■ 뺄셈

```
>>> 2-1
1
```

python 맛보기: 대화형 언어 경험하기

▪ 곱셈

```
>>> 3*4 Enter  
12
```

▪ 나눗셈

```
>>> 8/3 Enter  
2.6666666666666665
```

실습: IDLE 사용법

■ IDLE

- python을 설치하면 자동으로 같이 설치되는 프로그램
- 긴 python 프로그램 코딩에 적합

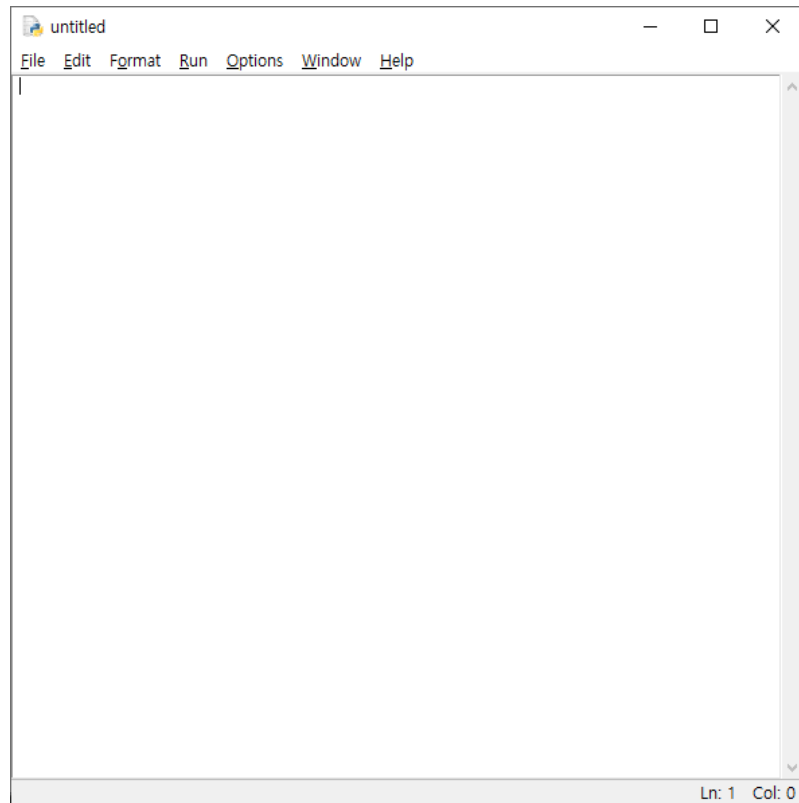
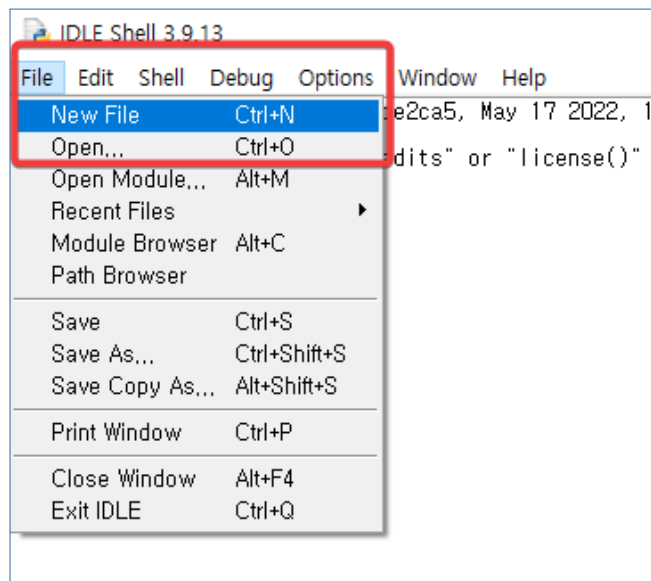
■ 실행

- windows 키를 누른 후 "idle" 입력, IDLE(Python 3.9 64-bit) 실행



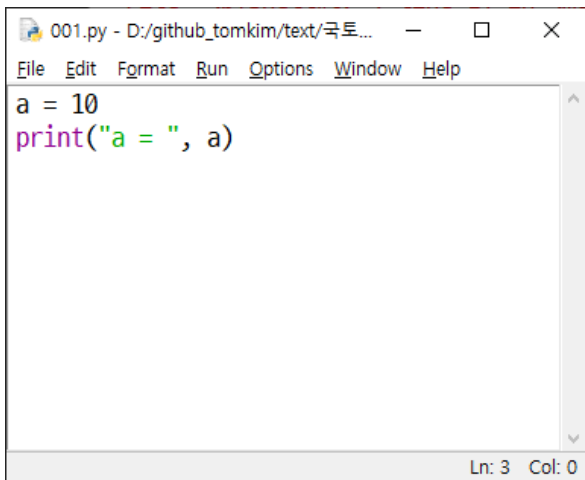
실습: IDLE 사용법

■ 새 파일



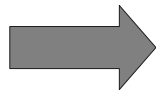
실습: IDLE 사용법

■ 실행: F5키



```
001.py - D:/github_tomkim/text/국토...  
File Edit Format Run Options Window Help  
a = 10  
print("a = ", a)  
Ln: 3 Col: 0
```

① 코드입력



② F5 키

```
===== RESTART: D:/py/001.py =====  
a = 10
```

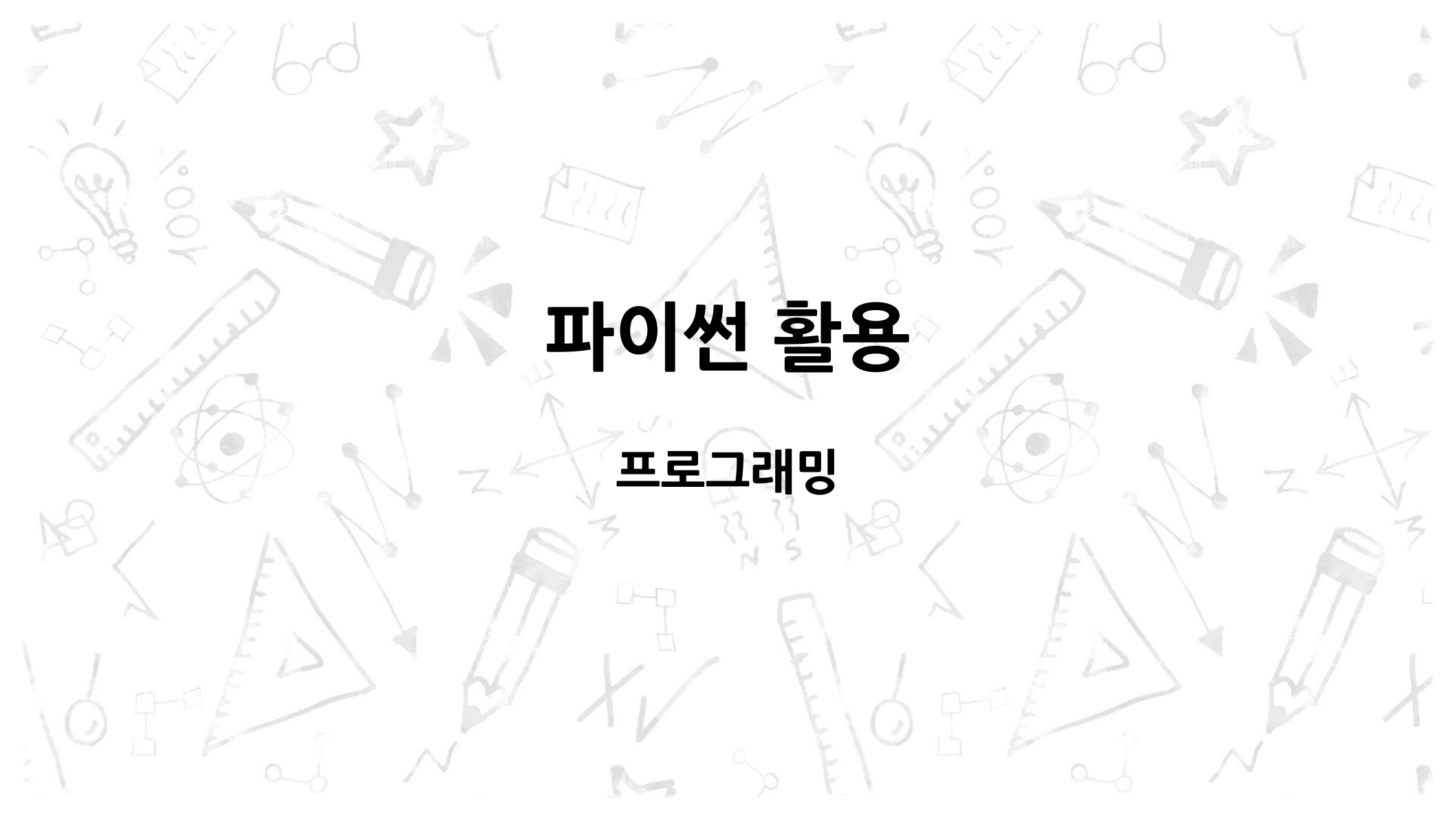
③ 실행 결과

정리

- **python은 인터프리터, 대화형 언어이다.**
 - 컴퓨터에게 정보를 주고 그에 대한 답을 받는 식이다.
 - 이런 특성으로 C언어 등에 비해 처리 속도가 느린 편이다.
 -
- **python은 무료이다.**
- **python으로 웹사이트 제작, AI, 데이터 분석 프로그래밍이 가능하다.**
- **python으로는 실시간 빠른 처리가 필요한 프로그램은 만들지 않는다.**

정리

- python은 비교적 쉬운 문법으로 초보자들이 쉽게 사용할 수 있다.
- python은 방대한 라이브러리를 이용해 빠르게 개발할 수 있다.
- 사용자가 많으니 라이브러리가 많이 만들어지고, 라이브러리가 많으니 사용자가 계속 유입되고, 유입된 사용자들이 또 라이브러리를 만들어 내는 선순환이 이루어지고 있다.



파이썬 활용

프로그래밍

프로그래밍이란

- 내가 원하는 일을 컴퓨터가 할 수 있도록 순서대로 적는 것
- 사람에게 일 시키는 것과 비슷하지만 훨씬 구체적으로 알려줘야 한다.
- 매우 비슷한 예가 "음식 레시피"
 1. 물 550cc를 냄비에 넣는다.
 2. 냄비 안의 물을 끓인다.
 3. 물이 끓으면 냄비 안에 스프를 넣는다.
 4. 바로 면을 넣는다.
 5. 5분간 기다린다.

컴퓨터에게 할 일을 알려주는 방법

- **프로그래밍 언어**

- 파이썬, C, 자바 등

- **파이썬**

- 주로 AI, 업무 자동화 등에 사용, 조금 느려도 괜찮은 프로그램
-

- **C언어**

- 주로 OS(windows11), 빠른 속도를 필요로 하는 프로그램, 핵심 기능, 파이썬

- **자바**

- 주로 웹서버 등 웹 관련

컴퓨터에게 뭘 시킬까

▪ 내가 하기 귀찮은 일

- 인간이 가장 싫어하는 일: 단순 반복 작업
- 컴퓨터가 가장 잘하는 일: 단순 반복 작업

} 업무자동화

▪ 규칙성이 있는 일

- 명확한 규칙성이 있는 일은 프로그래밍하기 쉽다.

▪ 정확함이 필요한 일

- 사람보다 훨씬 정확하고 빠르다.

코딩 vs 프로그래밍

- 그냥 혼용해서 써도 큰 문제 없음.
- 미세한 차이를 굳이 얘기하자면
 - 코딩: 프로그래밍 언어를 타이핑하는 행위 자체
 - 프로그래밍: 요구사항 정의, 설계, 코딩, 오류 수정 등 전체 과정

AI가 잘 하니 개발자는 더 이상 필요없다

- 코딩은 소프트웨어 개발의 일부일 뿐

- MS 어떤 팀에서는 지각한 개발자들에게 코딩을 시켰다. 다른 사람들은 설계 등에 집중

- 기술부채의 증가

- AI가 만들어준 코드로 소프트웨어 개발했는데 오류 발생 시 대처가 안되는 문제 증가
- 주변사항 고려없이 AI가 특정 방향의 코드만 작성해주는 문제

- 회사 요구에 의해 사직했던 개발자들이 다시 복귀하는 사례 발생 [\[기사\]](#)

- 개발자와 AI가 협력하는 시대로 나아가는 중

- 비개발자가 스스로 앱을 만들 수 있는 환경이 조성됨

AI 시대 비개발자가 앱 만들기

- 개발에 필요한 최소한의 지식만 배우자
- 복잡하고 어려운 코드는 AI가 만들어 준 것 사용
 - AI가 만들어 준 코드를 적용하고 실행할 수 있는 정도는 할 수 있어야
 - 실행환경은 사람이 만들어야 함.
 - 개발자들이 더하는 건 코드 리뷰 + 문제 발견 + 문제 수정 후 적용
- 아이디어만 있어도 앱 개발이 가능한 시대

컴퓨터는 0과 1만 안다는데

- 맞음. 전기가 통하고(1) 안통하고(0) 이것만 처리. 이것만 매우 빠르게 처리.
- 컴퓨터는 숫자만 다룬다.
 - 컴퓨터는 문자 모름
 - 컴퓨터는 그림 모름
 - 컴퓨터는 동영상, 음악 모두 모름. 그냥 숫자만 다룬다.

컴퓨터에게 문자란

- 문자는 사실 전부 숫자로 되어 있다.
- 숫자를 예쁜 글자 그림으로 화면에 배치해서 보여주는 것 - 폰트의 역할
 - 예: 숫자 65는 대문자 A로 예쁘게 보여준다

컴퓨터에게 그림이란

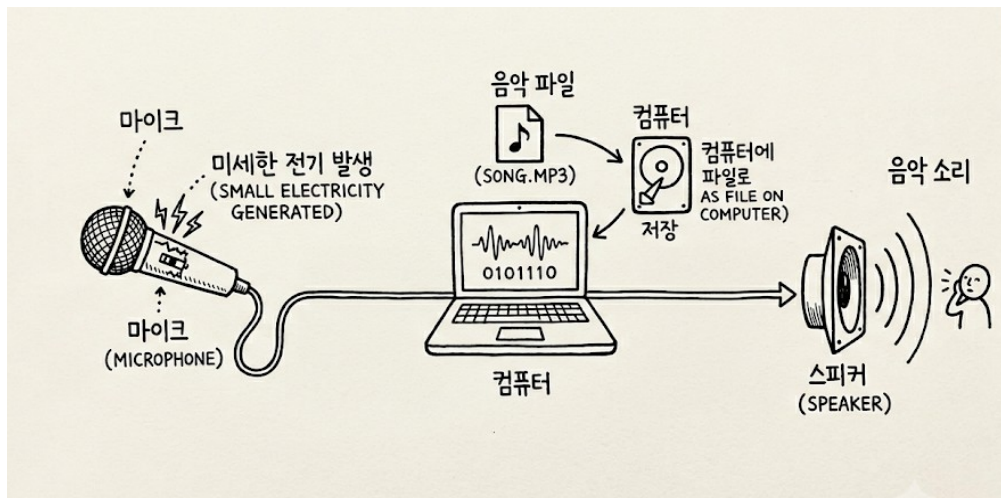
- 그림은 사실 전부 숫자로 되어 있다.
- 그림은 무수히 많은 점으로 이루어져 있다.
 - 예: 이 그림은 가로 1000개, 세로 600개의 점으로 이루어져 있다.
 - 예: 2026년 현재 일반적으로 쓰는 모니터는 가로 1920개, 세로 1080개의 점으로 구성
- 한 개의 점은 빨간색의 강도, 초록색의 강도, 파란색의 강도, 3개의 숫자로 구성

컴퓨터에게 동영상이란

- 전부 숫자로 된 그림을 연달아 붙여 놓은 것이라 동영상도 결국 숫자

컴퓨터에게 음악이란

- 음악은 사실 전부 숫자로 되어 있다.
- 음악 파일을 열어보면 왼쪽 채널, 오른쪽 채널에서 내야 할 소리 크기의 숫자값이 순서대로 적혀있다.



컴퓨터에서 관리하는 것은 모두 숫자

- 파일 확장자가 필요한 이유

- .hwp: 한컴 오피스 “한글”이 파일 안의 숫자를 이해해 문자로 만들어 보여줌
- .jpg: 이미지 뷰어가 파일 안의 숫자를 이해해 화면의 각 점에 색을 칠해 보여줌
- .mp3: 오디오 플레이어가 파일 안의 숫자를 이해해 시간에 맞게 스피커를 울려줌
- .mp4: 동영상 플레이어가 파일 안의 숫자를 이해해 시간에 맞게 연속된 이미지 보여줌

GPU

- **GPU: Graphics Processing Unit**

- 빠른 속도로 그래픽 관련 처리를 해주는 장치

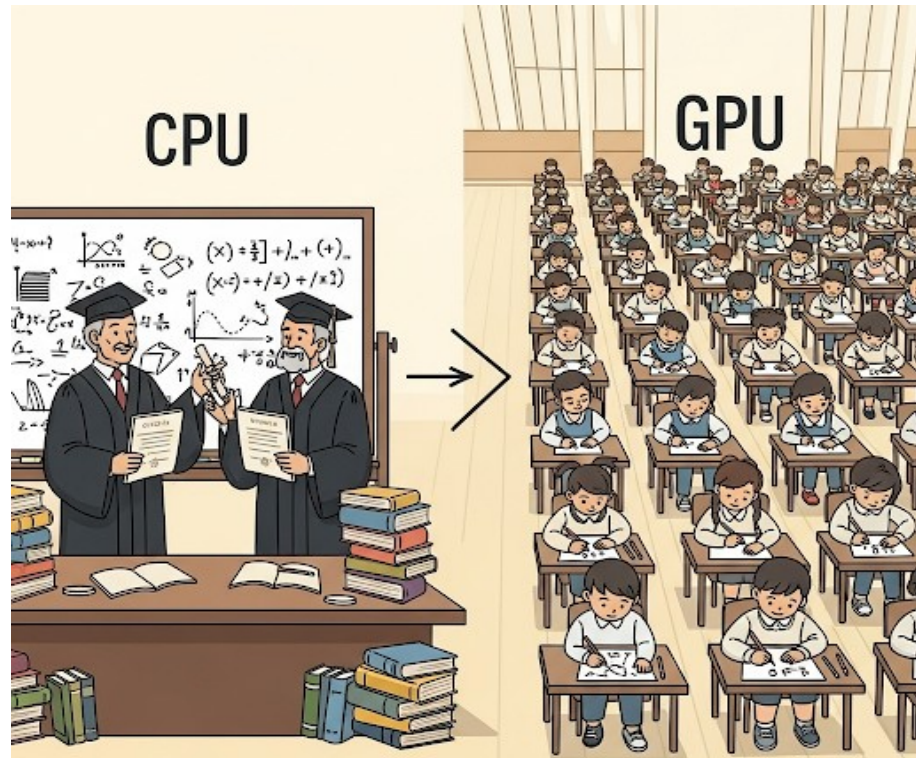


출처: nVidia 공식 홈페이지

GPU

■ CPU vs GPU

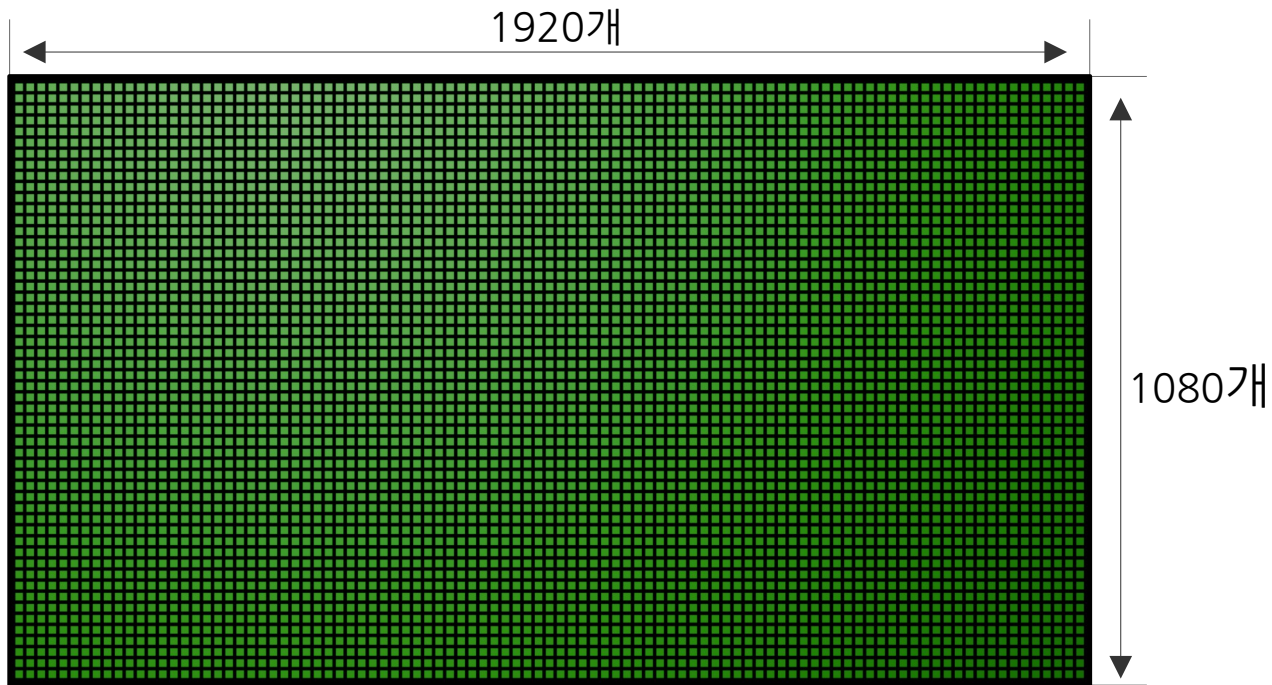
- CPU의 1개 일꾼: 어려운 문제 해결용.
- CPU 1개에 보통 1개~16개 일꾼 내장
- GPU의 1개 일꾼: 쉬운 문제 해결용.
- GPU 1개에 보통 몇 천~ 1만의 일꾼 내장



GPU

- 한 화면의 점(픽셀) 수

- 현재 일반적인 모니터의 점의 수: $1,920 * 1,080 = 2,073,600$ 개



GPU

■ 모니터는 1초에 몇 번 새로 고침 하나: 60회

- $1,920 * 1,080 = 2,073,600$. $2,073,600$ 개의 점 * 60회 = 124,416,000



1초에 124,416,000개의 점을
계산해서 보여줘야 함.

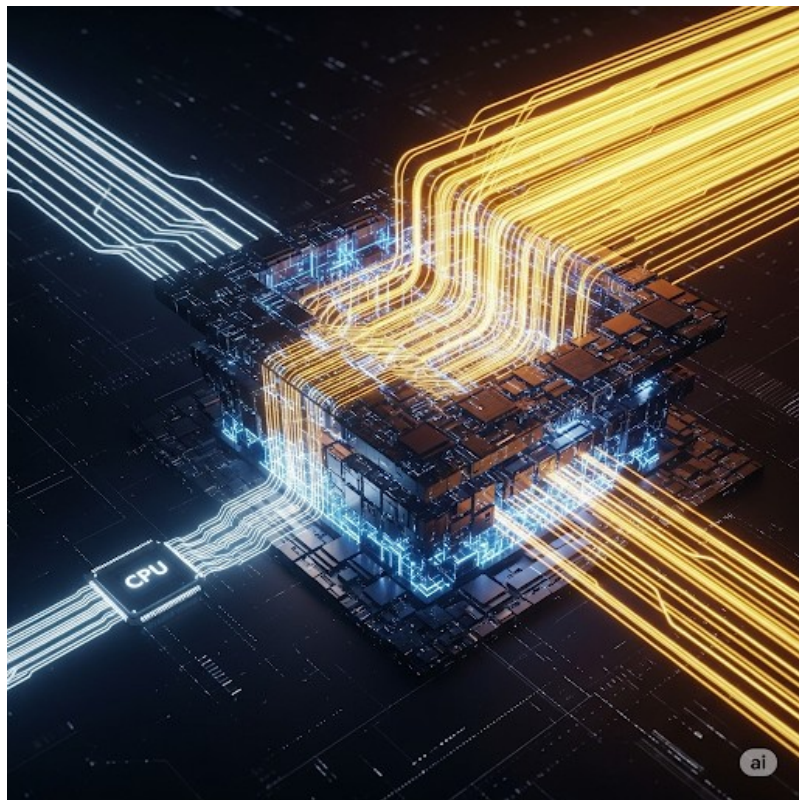
GPU는 **동시에** 몇 천개의 일꾼을
이용해 점의 색을 빠르게 계산한다.

점의 색 계산은 복잡하지 않고
단순한 계산이다. 이걸 초당
1억 2천여개의 점에 대해 처리하니
CPU보다 GPU가 더 적합

GPU

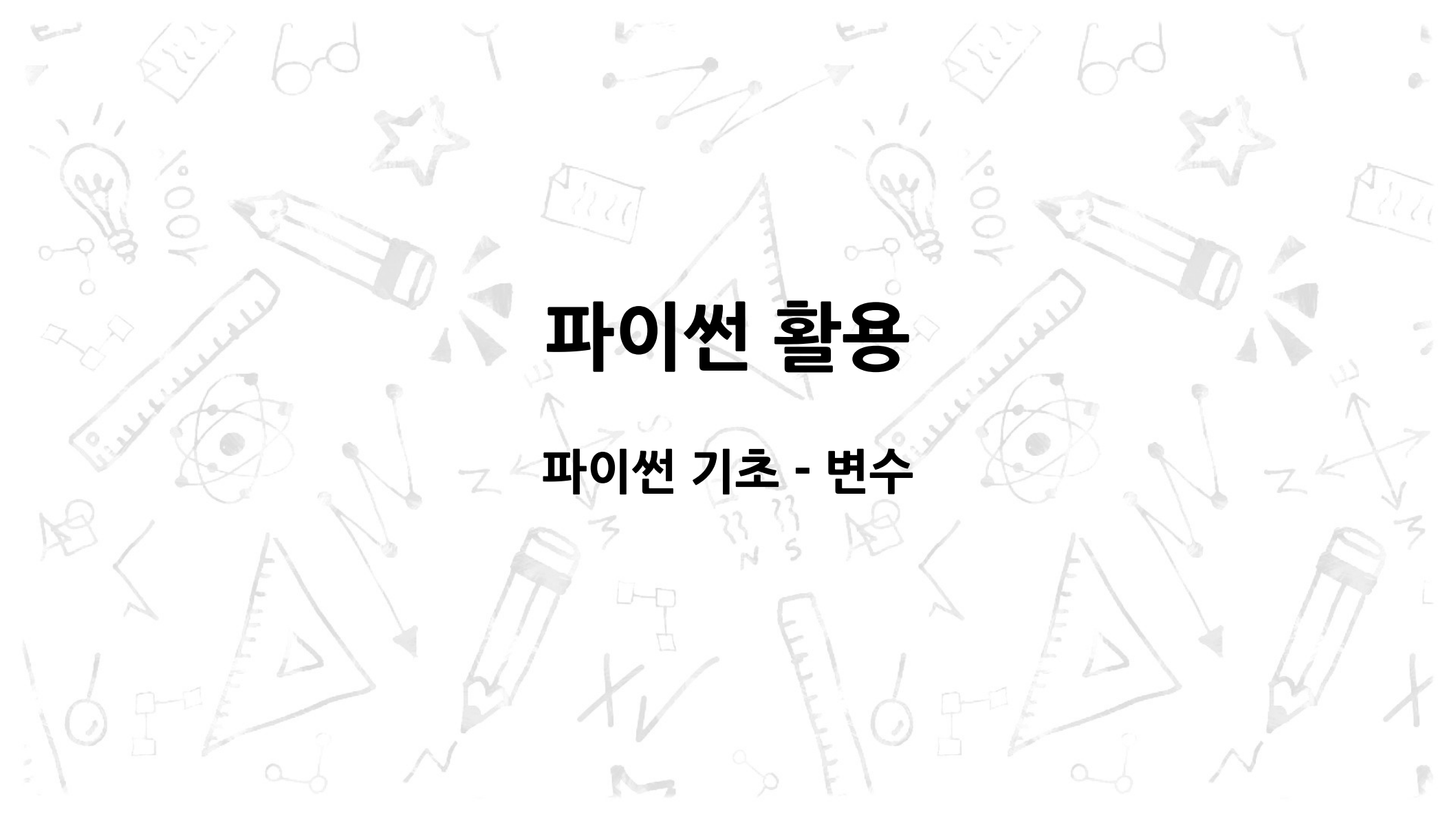
▪ AI를 위해 GPU가 사용되는 이유

- 복잡하지는 않지만 많은 양의 자료를 처리해야 하기 때문
- AI에서 처리할 작업을 GPU가 동시에 처리하기 쉽도록 발전해 옴
- AI를 위해 GPU가 많이 사용되다 보니 GPU에서 AI를 위한 전용 기능도 추가함
- 이제 AI와 GPU는 서로 보완하며 발전하는 관계



개발 역량을 기반으로 사업 기회 창출

- 개발 역량이 있으면 작업을 대하는 시선이 달라진다
 - 새로운 소프트웨어 개발 아이디어
 - 현재 하는 일을 더 빠르고 정확하게 할 아이디어
 - 현재 하는 귀찮은 일을 자동으로 처리해 줄 아이디어

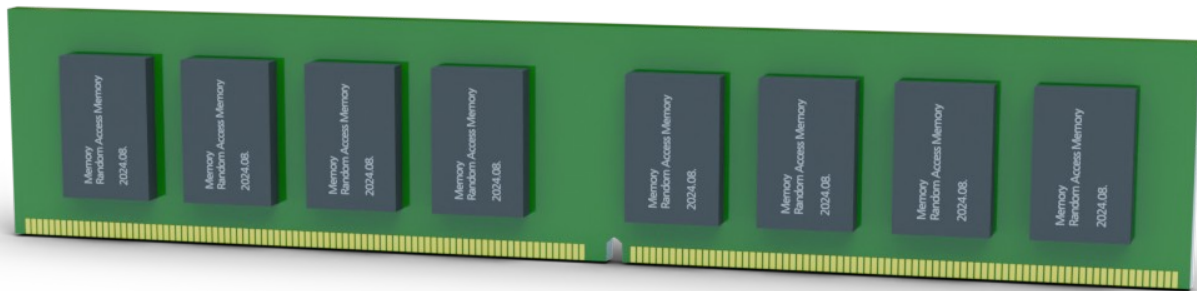


파이썬 활용

파이썬 기초 - 변수

변수(variable)

- [이 사업에는 "변수"가 많아서 너무 힘들어]
 - 변수: 변하는 숫자, 변할 수 있는 숫자
- 프로그래밍 영역에서 변수: 메모리 내에 자료 하나가 저장되는 곳
 - 기억할 일이 있을 때 사람은 종이에 적고, 컴퓨터는 메모리에 적는다



변수(variable)

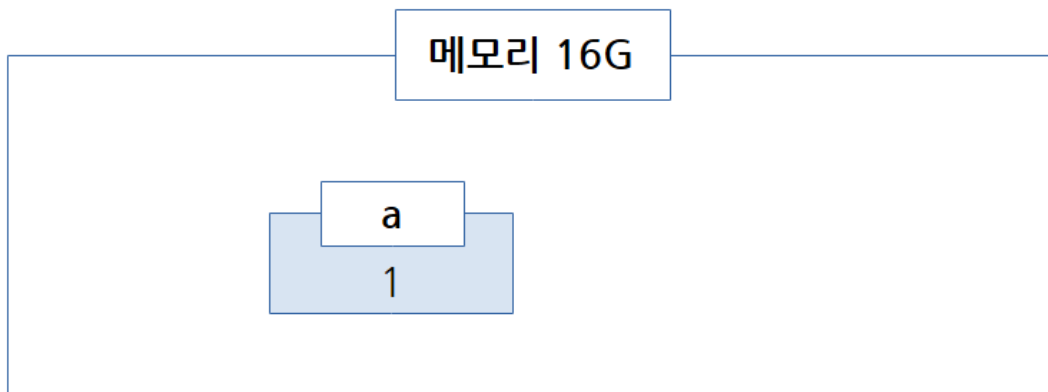
$$a = 1$$

변수(variable)

$$\begin{array}{ccc} a & = & 1 \\ \hline \text{변수 이름} & & \text{변수에 담을 값} \end{array}$$

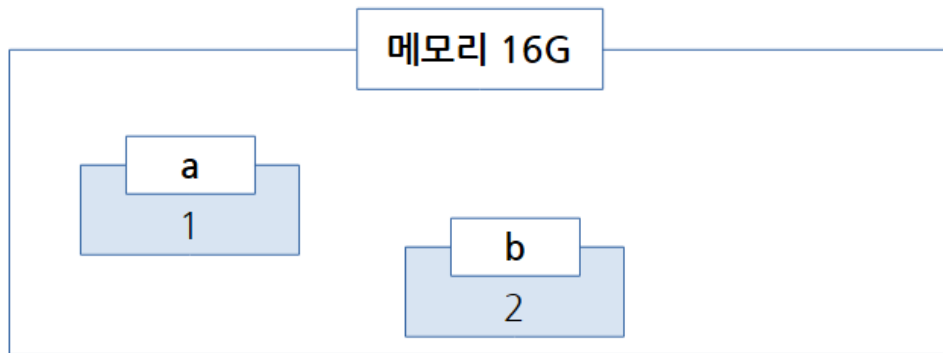
변수(variable)

$a = 1$

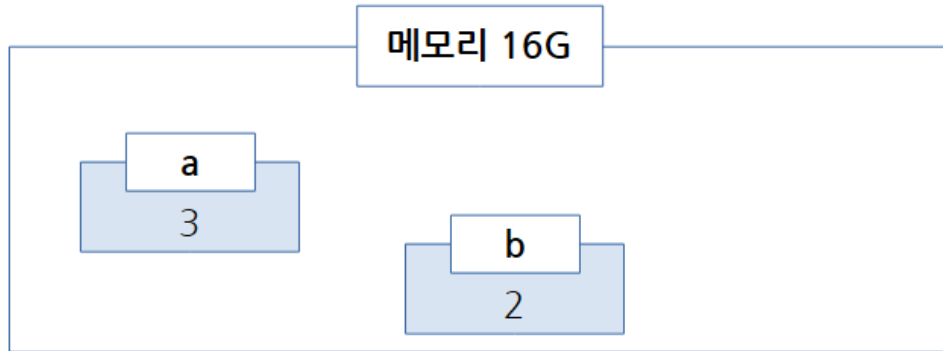


변수(variable)

```
>>> a = 1  
>>> b = 2
```



```
>>> a = 3
```

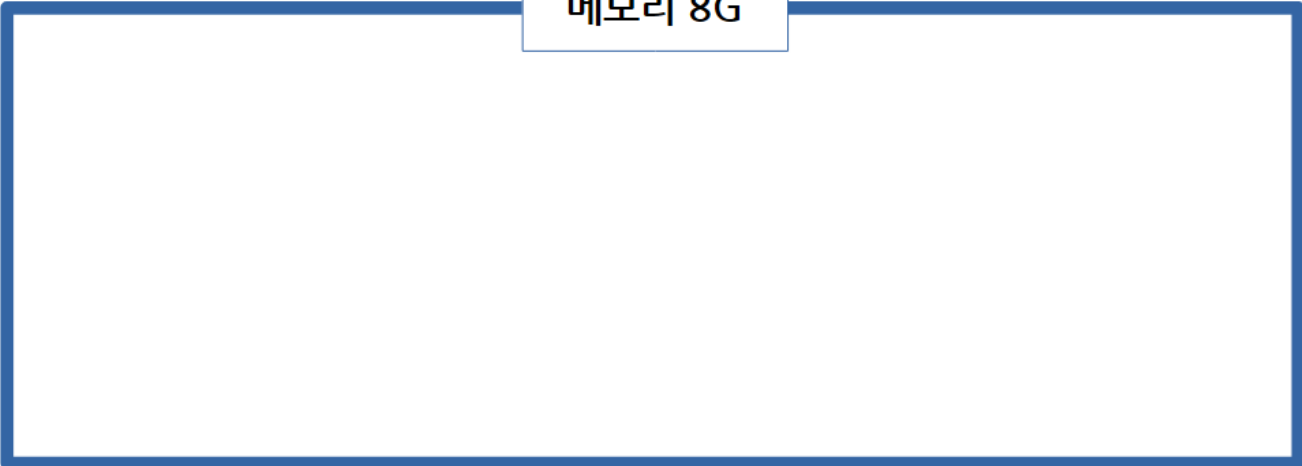


실습

- 코드가 실행되면 메모리에 어떻게 구성되는지 오른쪽 메모리에 그려보기

```
>>> my = 15  
>>> your = -5  
>>> apple = 10
```

메모리 8G



실습: 숫자형

```
>>> 1 + 1
```

```
2
```

```
>>> 1 + 1.1
```

```
2.1
```

```
>>> -5 * 3
```

```
-15
```

숫자형

- 수를 계산하기 위한 자료형
- 정수(integer)
 - ... , -3, -2, -1, 0, 1, 2, 3, ...
- 실수
 - 소수점으로 표현되는 수
 - 3.14, 53.12, -0.15 등

숫자형 변수를 이용한 연산

$$a = 1$$

$$b = a + 2.1$$

- a는 정수형 변수일까 실수형 변수일까?
- b는 정수형 변수일까 실수형 변수일까?

변수에 저장된 값 보기

```
>>> a = 1
>>> b = a + 2.1
>>> a
1
>>> b
3.1
```

* 변수에 저장된 값을 보려면 그냥 그 **변수명을 입력하고 엔터키**를 치면 된다.

변수에 저장된 값 보기

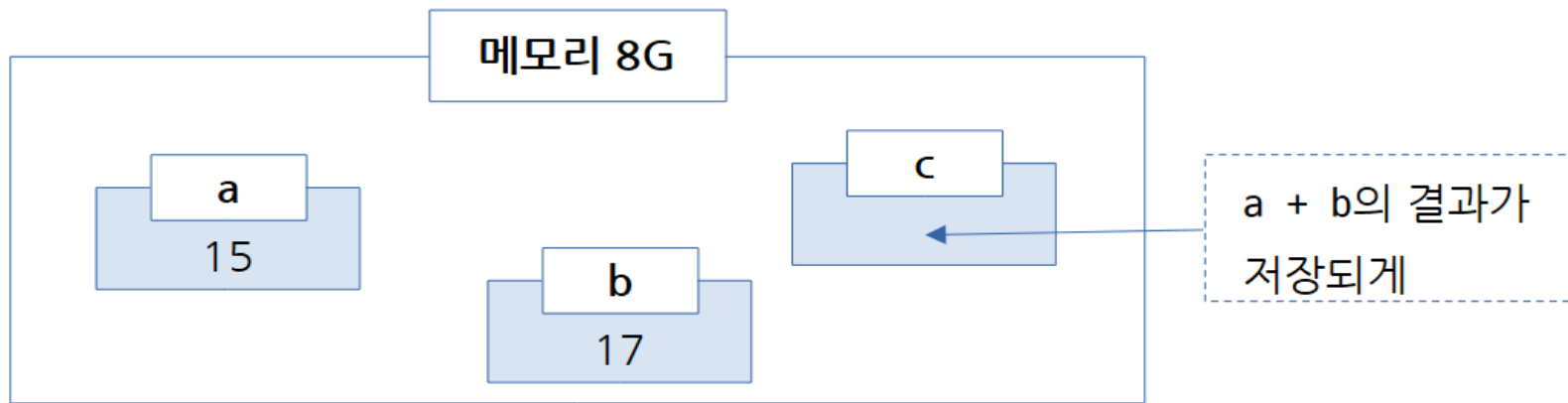
```
>>> a = 1
>>> b = a + 2.1
>>> print(a)
1
>>> print("b = ", b)
b = 3.1
>>> print("a = ", a, "b = ", b)
a = 1 b = 3.1
```

print 함수(기능)을 이용해 화면에 표시할 수 있다.

coma(,)를 이용해 여러 개의 값을 한 줄에 표시할 수 있다.

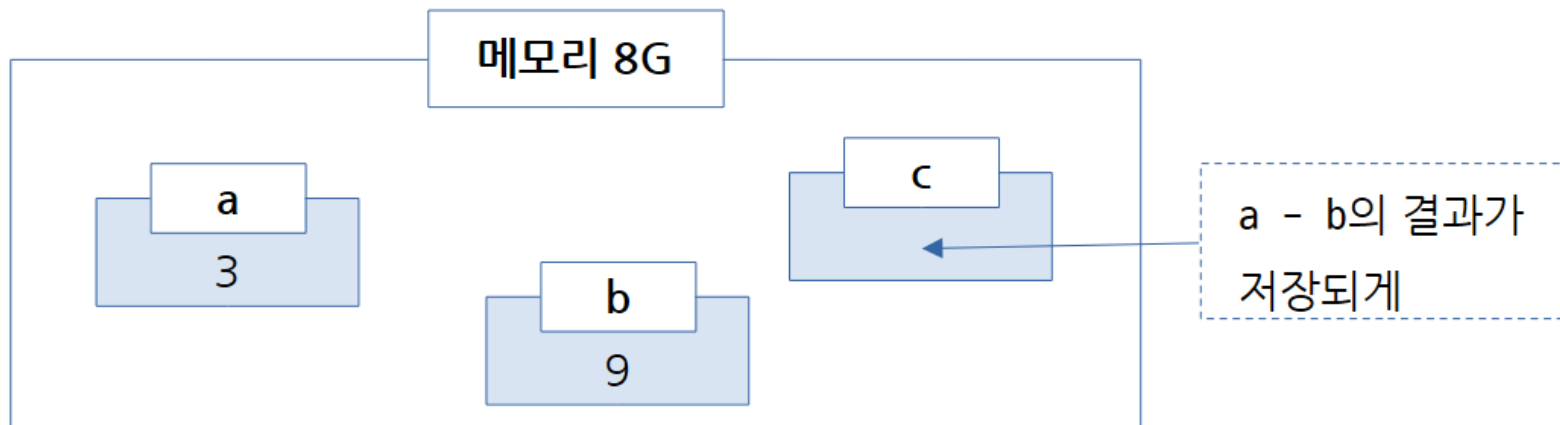
실습: 숫자형 변수를 이용한 연산

- python을 이용해 다음의 숫자 연산을 해 보자.



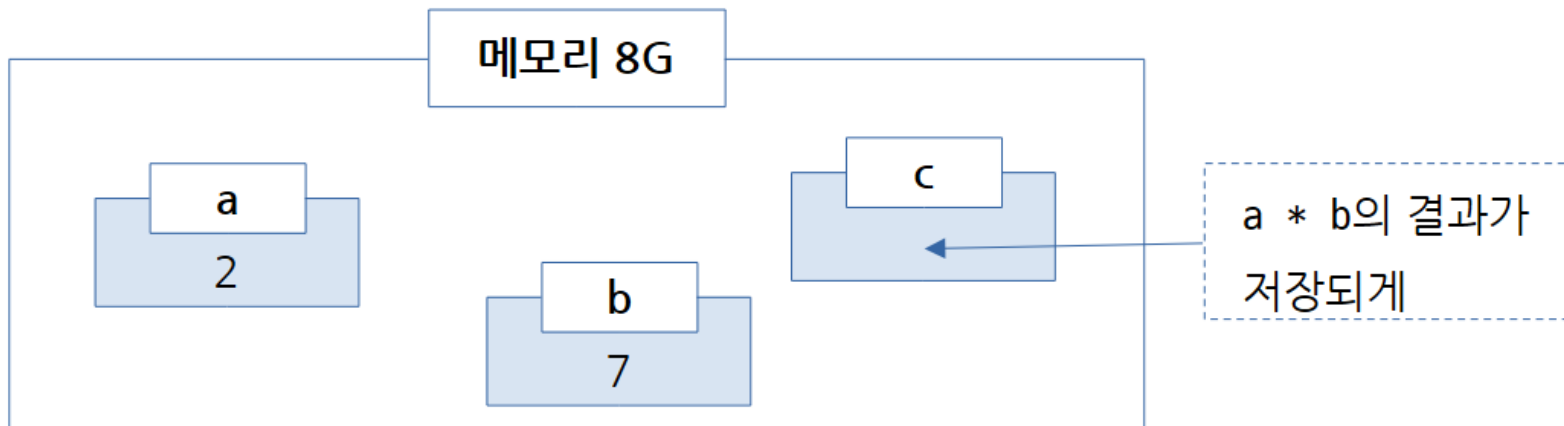
실습: 숫자형 변수를 이용한 연산

- python을 이용해 다음의 숫자 연산을 해 보자.



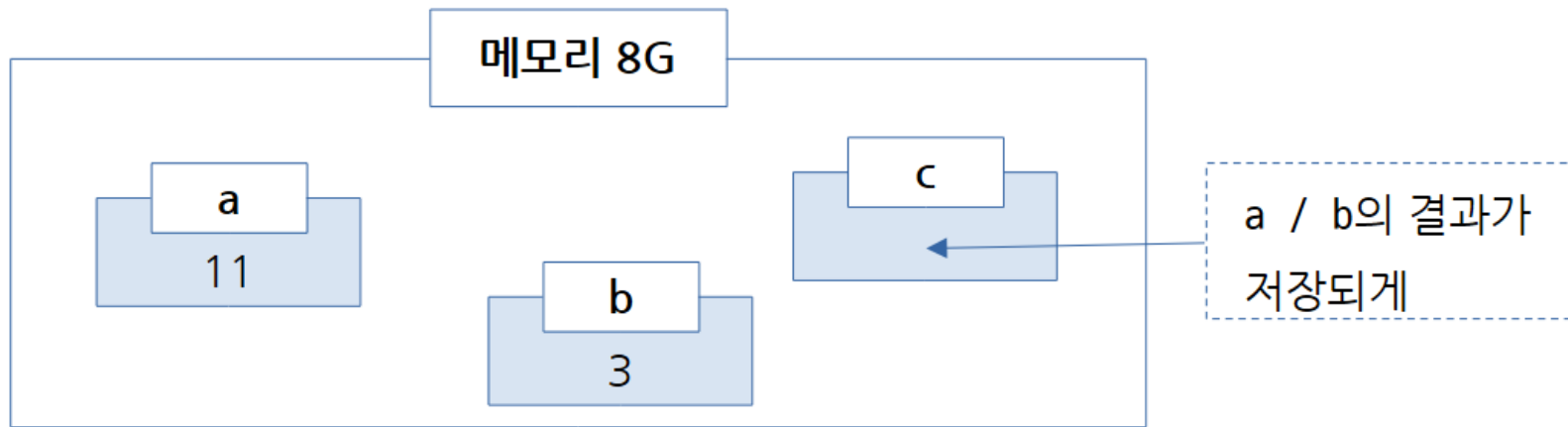
실습: 숫자형 변수를 이용한 연산

- python을 이용해 다음의 숫자 연산을 해 보자.



실습: 숫자형 변수를 이용한 연산

- python을 이용해 다음의 숫자 연산을 해 보자.



숫자 연산을 위한 다양한 연산자

- **// 연산자: 나눗셈의 몫**

```
>>> 11 / 3  
3.6666666666666665
```

```
>>> 11 // 3  
3
```

$$\begin{array}{r} 3 \leftarrow \text{몫} \\ 3 \overline{) 11} \\ \underline{9} \\ 2 \leftarrow \text{나머지} \end{array}$$

숫자 연산을 위한 다양한 연산자

- **%** 연산자: 나눗셈의 나머지

```
>>> 11 / 3  
3.6666666666666665
```

```
>>> 11 % 3  
2
```

$$\begin{array}{r} 3 \overline{) 11} \\ \underline{9} \\ 2 \end{array}$$

← 몫

← 나머지

숫자 연산을 위한 다양한 연산자

- ****** 연산자: 제곱

```
>>> 2 ** 3  
8
```

$$2 ** 3 = 2^3 = 2 * 2 * 2$$

문자열

- 문자열: 문자의 나열

- "apple"
- "안녕"
- 'tomato'
- '반가워'

- 숫자와 달리 문자열은 앞뒤를 큰 따옴표나 작은 따옴표로 감싸야 한다.

문자열을 담은 변수

```
>>> a = "안녕"
```

```
>>> a
```

```
'안녕'
```

```
>>> a = "Son's house"
```

```
>>> a
```

```
"Son's house"
```

```
>>> a = '저는 "홍길동"입니다'
```

```
>>> a
```

```
'저는 "홍길동"입니다'
```

문자열 연산 +

```
>>> a = "나의 이름은"
```

```
>>> b = "홍길동"
```

```
>>> c = a + b
```

```
>>> c
```

```
'나의 이름은홍길동'
```

실습: 문자열 연산 +

```
>>> a = "나의 이름은"
```

```
>>> b = "홍길동"
```

```
>>> c = a + b
```

```
>>> c
```

```
'나의 이름은홍길동'
```

왼쪽 코드의 첫 번째 줄만 수정해

나의 이름은 **홍**길동

과 같이 홍길동 앞에
빈 칸이 나오도록 하시오.

실습: 문자열 연산 +

```
>>> a = "나의 이름은"
```

```
>>> b = "홍길동"
```

```
>>> c = a + b
```

```
>>> c
```

```
'나의 이름은홍길동'
```

왼쪽 코드의 두 번째 줄만 수정해

나의 이름은 **홍**길동

과 같이 홍길동 앞에
빈 칸이 나오도록 하시오.

실습: 문자열 연산 +

```
>>> a = "나의 이름은"
```

```
>>> b = "홍길동"
```

```
>>> c = a + b
```

```
>>> c
```

```
'나의 이름은홍길동'
```

왼쪽 코드의 세 번째 줄만 수정해

나의 이름은 **홍**길동

과 같이 홍길동 앞에
빈 칸이 나오도록 하시오.

python에서 제공하는 문자열 기능들

<https://docs.python.org/ko/3/library/stdtypes.html#string-methods>



참, 거짓 타입

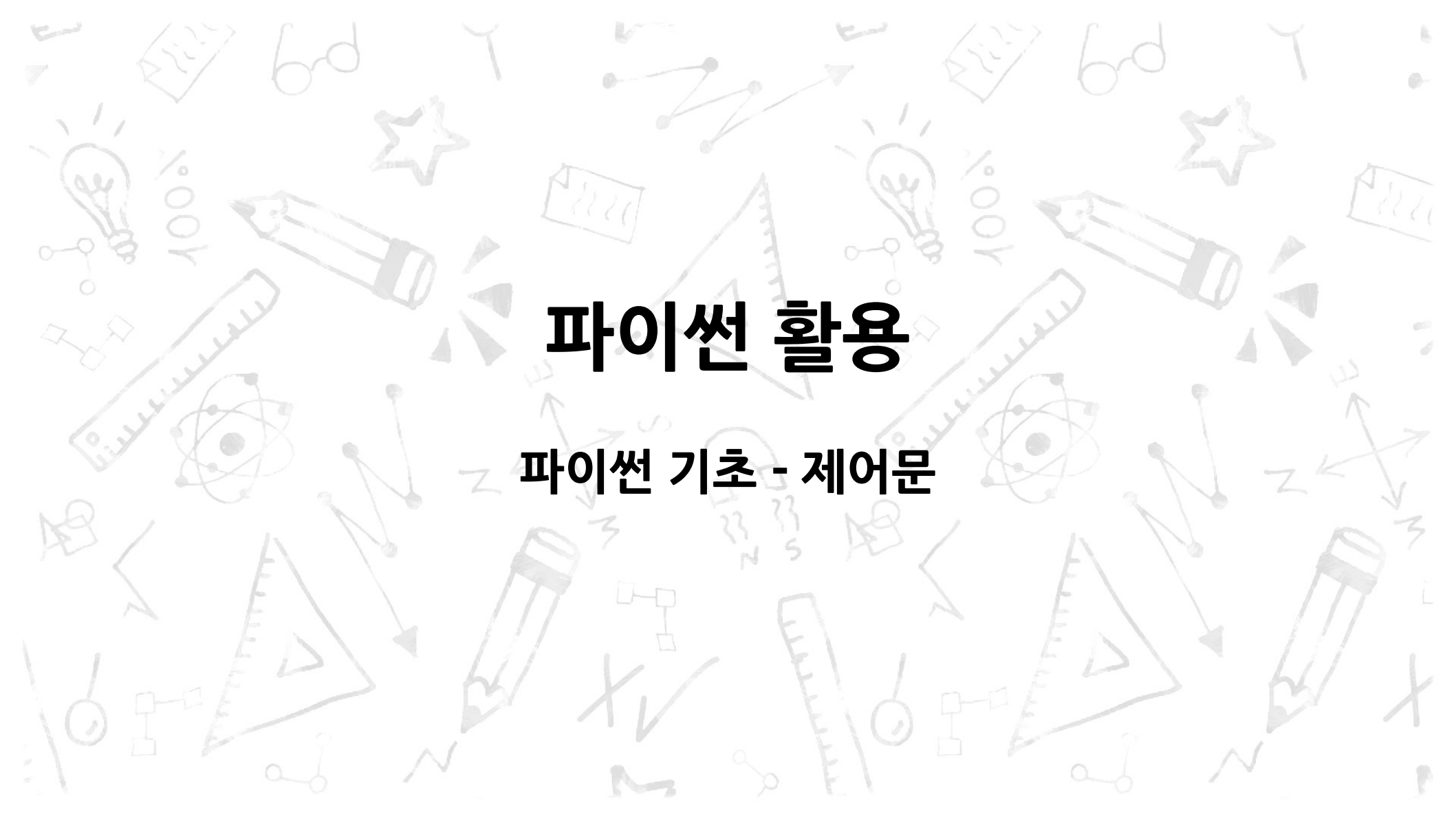
- 보통 bool 형이라고 부름.
 - 사용자가 입력한 값이 참인지 거짓인지 판단
 - 참: True
 - 거짓: False
 - True, False는 대소문자 구분함.

```
>>> 1 < 3
```

```
True
```

```
>>> 3 < 1
```

```
False
```



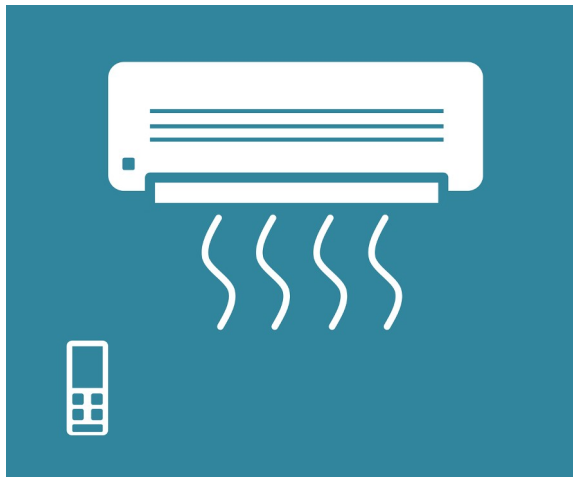
파이썬 활용

파이썬 기초 - 제어문

제어

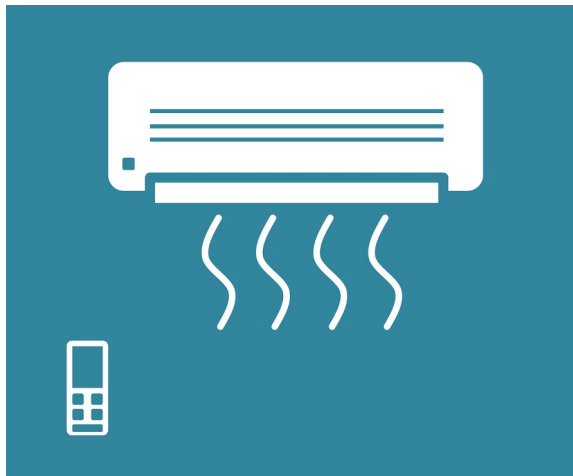
- control
- 주로 기계나 시스템을 통제, 조절하는 것을 뜻하는 한자어(制御)
- 프로그램의 흐름을 제어하기 위해 python은 다음을 제공함
 - 조건문
 - 반복문

조건문



- 에어컨의 온도를 26도로 설정하면,
 - 현재 온도 33도
 - 온도가 32, 31, 30... 27도로 내려갈 때 까지 에어컨이 냉기 발산
 - **설정 온도인 26도가 되면** 에어컨 냉기 발산 중지
 - 다시 27도로 올라가면 에어컨이 냉기를 발산하며 온도를 낮추기 위해 노력함.

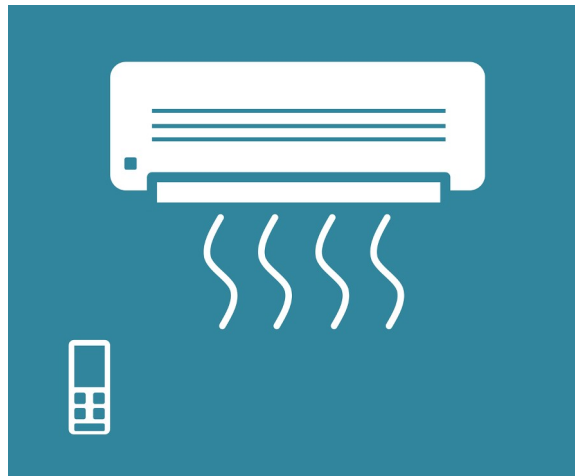
조건문



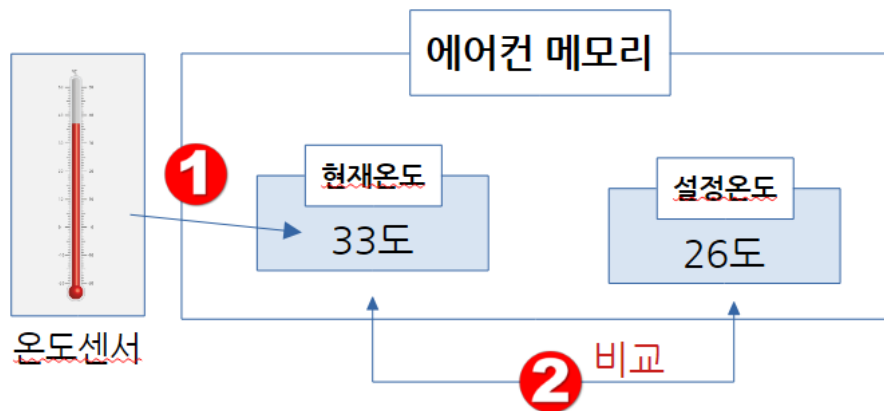
■ 에어컨 내부의 작은 컴퓨터가 하는 일

- 온도 센서에서 주기적으로(예를 들면 1초에 한 번씩) 온도를 가져옴 => temp 변수에 저장
- temp 변수의 값과 사용자가 설정한 온도(26도) **비교**
- 현재 온도(temp)가 사용자가 설정한 온도보다 **크면** 에어컨의 냉기 발산 계속
- 현재 온도(temp)가 사용자가 설정한 온도와 **같거나 작으면** 에어컨의 냉기 발산 중지

조건문



■ 에어컨 내부의 작은 컴퓨터가 하는 일



- 온도센서에서 값을 가져와 설정온도와 **비교**하는 일을 끊임없이 반복한다.

조건문

- 에어컨 온도조절 동작 코드

```
>>> 현재온도 = 33
>>> 설정온도 = 26
>>> if 현재온도 <= 설정온도 :
...     print("냉기 발산을 멈춥니다")
```


조건문

▪ if 문의 syntax

```
if 조건식 :
```

```
    수행할 내용 1
```

```
    수행할 내용 2
```

tab키를 눌러 들여쓰기를 해야 한다.

조건식이 참일 때만,

수행할 내용 1, 수행할 내용 2를 실행한다.

* 조건식이 참 / 거짓인 예:

a = 3 일 때,

if a < 5 # 3 < 5 가 되어 조건식이 참

if a > 5 # 3 > 5 가 되어 조건식은 거짓

조건문

▪ if 문의 들여쓰기

```
if 조건식 :
```

```
    a = 1           ①
```

```
    b = 2           ②
```

```
c = 3              ③
```

왼쪽 if 문의 경우,

조건식이 참이기 때문에 실행되는 것은,
조건식이 참이기 때문에 영향을 받는 것은,

들여쓰기를 한 ①과 ② 이다.

③은 if 문과 상관없이 항상 실행된다.

조건문 활용의 다양한 예

▪ 총 싸움 게임

- 적이 쏜 총알의 위치와 내 몸통의 위치를, 3차원 공간정보를 **비교**해서 매우 근접하면 총알에 맞은 것으로 판단해 에너지를 줄임

▪ 칼 싸움 게임

- 적이 휘두른 칼의 위치와 내 몸통의 위치를, 3차원 공간정보를 **비교**해서 매우 근접하면 칼에 베인 것으로 판단해 에너지를 줄임

▪ 시계 알람

- 현재 시각과 사용자가 설정한 알람 시각을 **비교**해서 두 값이 같아지는 순간 알람을 시작함.

조건문 활용의 다양한 예

- 지각한 사람 찾아내기

- ERP에 기록된 출근 시각을 09:00 과 비교해 09:00보다 크면 지각으로 판단

- 기타 등등 매우 많은 곳, 거의 모든 프로그램에서 사용됨.

- 조건문은 프로그램의 흐름을 바꾸는 매우 중요한 요소이다.

조건식

- 조건문의 조건식

if 설정온도 <= 현재온도 :

조건식

조건식

▪ 비교 연산자

- 크다: $a < b$ b 가 a 보다 크다
- 작다: $a > b$ b 가 a 보다 작다
- 크거나 같다: $a \leq b$ b 가 a 보다 크거나 같다
- 작거나 같다: $a \geq b$ b 가 a 보다 작거나 같다
- 같다: $a == b$ a 와 b 가 같다
- 같지 않다: $a != b$ a 와 b 가 같지 않다

실습: 비교연산자

```
>>> 3 < 5
```

```
True
```

```
>>> 3 > 5
```

```
False
```

```
>>> 3 <= 5
```

```
True
```

```
>>> 3 >= 5
```

```
False
```

```
>>> 3 == 5
```

```
False
```

```
>>> 3 != 5
```

```
True
```

실습: 비교연산자를 이용한 if 문

```
if 3 < 5 :  
    print ("3 < 5가 참이면 이 문장이 표시됨")  
  
if 3 > 5 :  
    print ("3 > 5가 참이면 이 문장이 표시됨")  
  
if 3 <= 5 :  
    print ("3 <= 5가 참이면 이 문장이 표시됨")  
  
if 3 >= 5 :  
    print ("3 >= 5가 참이면 이 문장이 표시됨")  
  
if 3 == 5 :  
    print ("3 == 5가 참이면 이 문장이 표시됨")  
  
if 3 != 5 :  
    print ("3 != 5가 참이면 이 문장이 표시됨")
```



3 < 5가 참이면 이 문장이 표시됨
3 <= 5가 참이면 이 문장이 표시됨
3 != 5가 참이면 이 문장이 표시됨

실습: 비교연산자를 이용한 if 문

▪ 에어컨 온도조절 동작 코드

```
현재온도 = 33
```

```
설정온도 = 26
```

```
if 현재온도 <= 설정온도 :
```

```
    print("냉기 발산을 멈춥니다.")
```

현재온도 = 33 일 때 F5 눌러 실행,

현재온도 = 32 로 수정하고 F5 눌러 실행,

.

.

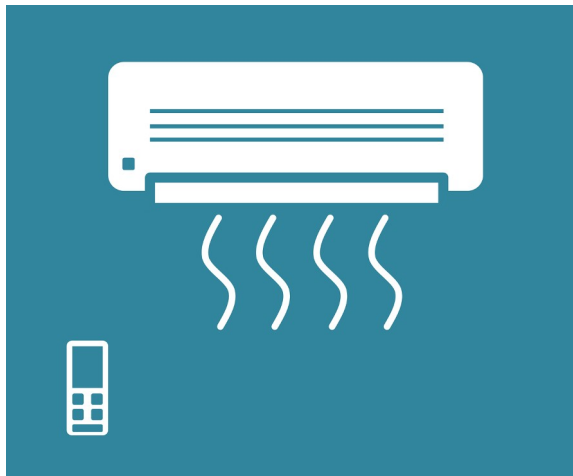
.

현재온도 = 28 로 수정하고 F5 눌러 실행,

현재온도 = 27 로 수정하고 F5 눌러 실행,

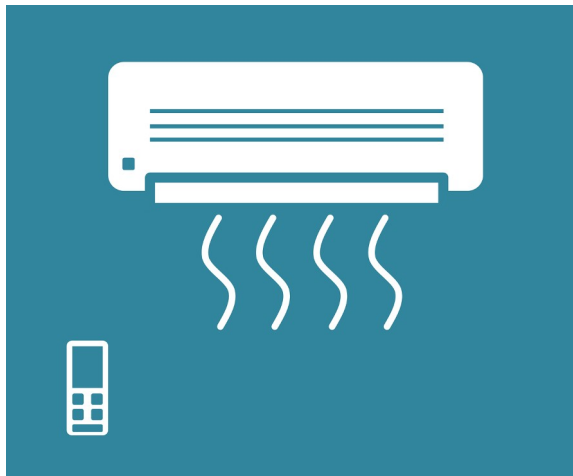
현재온도 = 26 로 수정하고 F5 눌러 실행.

조건문



- 에어컨의 온도를 26도로 설정하면,
 - 현재 온도 33도
 - 온도가 32, 31, 30... 27도로 내려갈 때 까지 에어컨이 냉기 발산
 - 설정 온도인 26도가 되면 에어컨 냉기 발산 중지
 - 다시 27도로 올라가면 에어컨이 냉기를 발산하며 온도를 낮추기 위해 노력함.

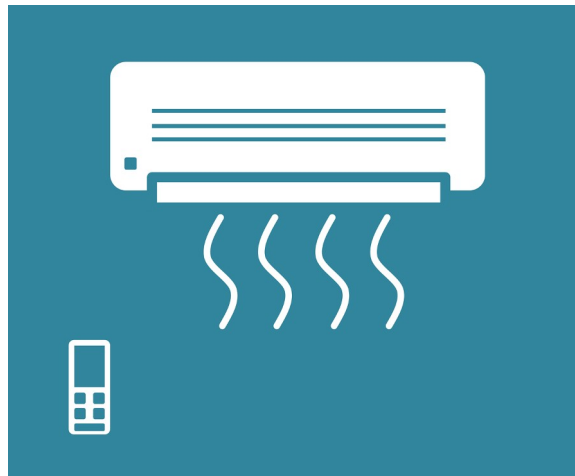
조건문



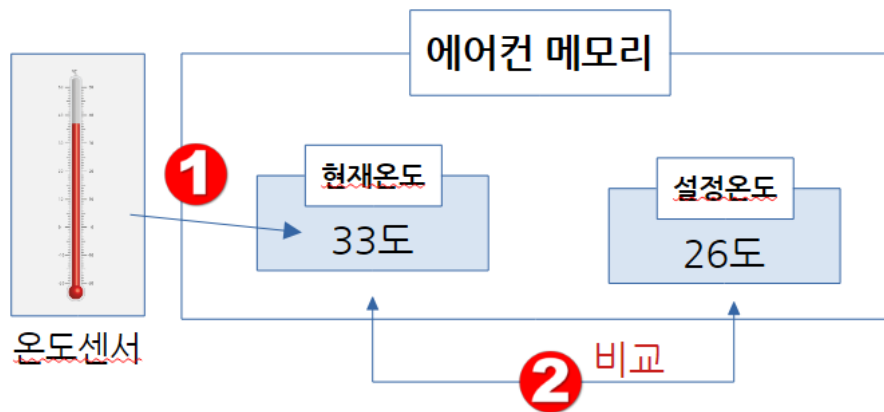
■ 에어컨 내부의 작은 컴퓨터가 하는 일

- 온도 센서에서 **주기적으로** (예를 들면 1초에 한 번씩) 온도를 가져옴 => temp 변수에 저장
- temp 변수의 값과 사용자가 설정한 온도(26도) 비교
- 현재 온도(temp)가 사용자가 설정한 온도보다 크면 에어컨의 냉기 발산 계속
- 현재 온도(temp)가 사용자가 설정한 온도와 같거나 작으면 에어컨 냉기 발산 중지

반복문



■ 에어컨 내부의 작은 컴퓨터가 하는 일



- 온도센서에서 값을 가져와 설정온도와 비교하는 일을 끊임없이 반복한다.

반복문 - while

```
a = 0
```

```
while a < 3 :
```

```
    print (a)
```

```
    a = a + 1
```



```
0
```

```
1
```

```
2
```

tab키를 눌러 들여쓰기를 해야 한다.

반복문 - while

```
a = 0
```

```
while a < 3 :  
    print (a)  
    a = a + 1
```

while: ~하는 동안에

while a < 3 : a가 3 미만인 동안에

반복문 - while의 조건식

```
a = 0
```

```
while a < 3 :
```

```
    print (a)
```

```
    a = a + 1
```

while: ~하는 동안에

조건식이 참(True)인 동안에

반복문 - while의 조건식

```
a = 0
```

```
while True :
```

```
    print (a)
```

```
    a = a + 1
```

무한반복

조건 자체가 True라 항상 조건식이 참이기 때문에 전원을 끄기 전까지 무한히 반복됨.

반복문 - while문을 사용한 에어컨 온도 조절

```
설정온도 = 26
```

```
while True:
```

```
    현재온도 = get_temp()
```

```
        if 현재온도 <= 설정온도 :
```

```
            print("냉기 발산을 멈춥니다.")
```

```
        else :
```

```
            print("냉기를 발산합니다.")
```

무한반복

조건식이 항상 참이기 때문에 전원을 끄기 전까지
다음의 실행을 반복함

"온도센서의 온도를 가져와서
사용자의 설정값과 비교해서
그 결과에 따라 에어컨을 끄거나 가동함"

* get_temp()는 온도센서에서 온도를 가져오는 기능. 여기서는 구현하지 않는다.

실습: 반복문 - while

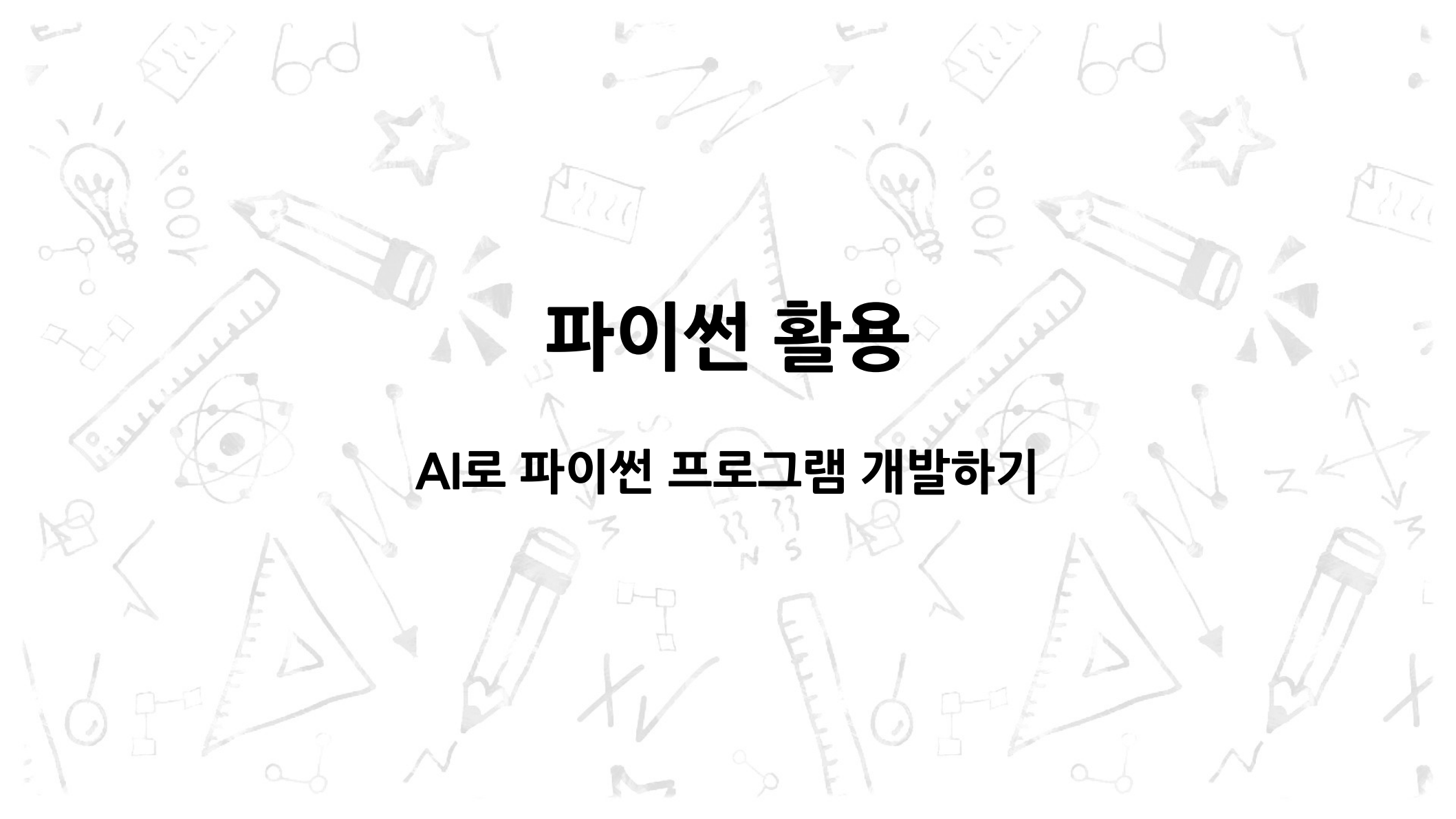
- 1, 3, 5, 7, 9, ... 와 같이 홀수만 99까지 출력

```
a = 1
```

```
while a <= 99 :
```

```
    print(a)
```

```
    a = a + 2
```

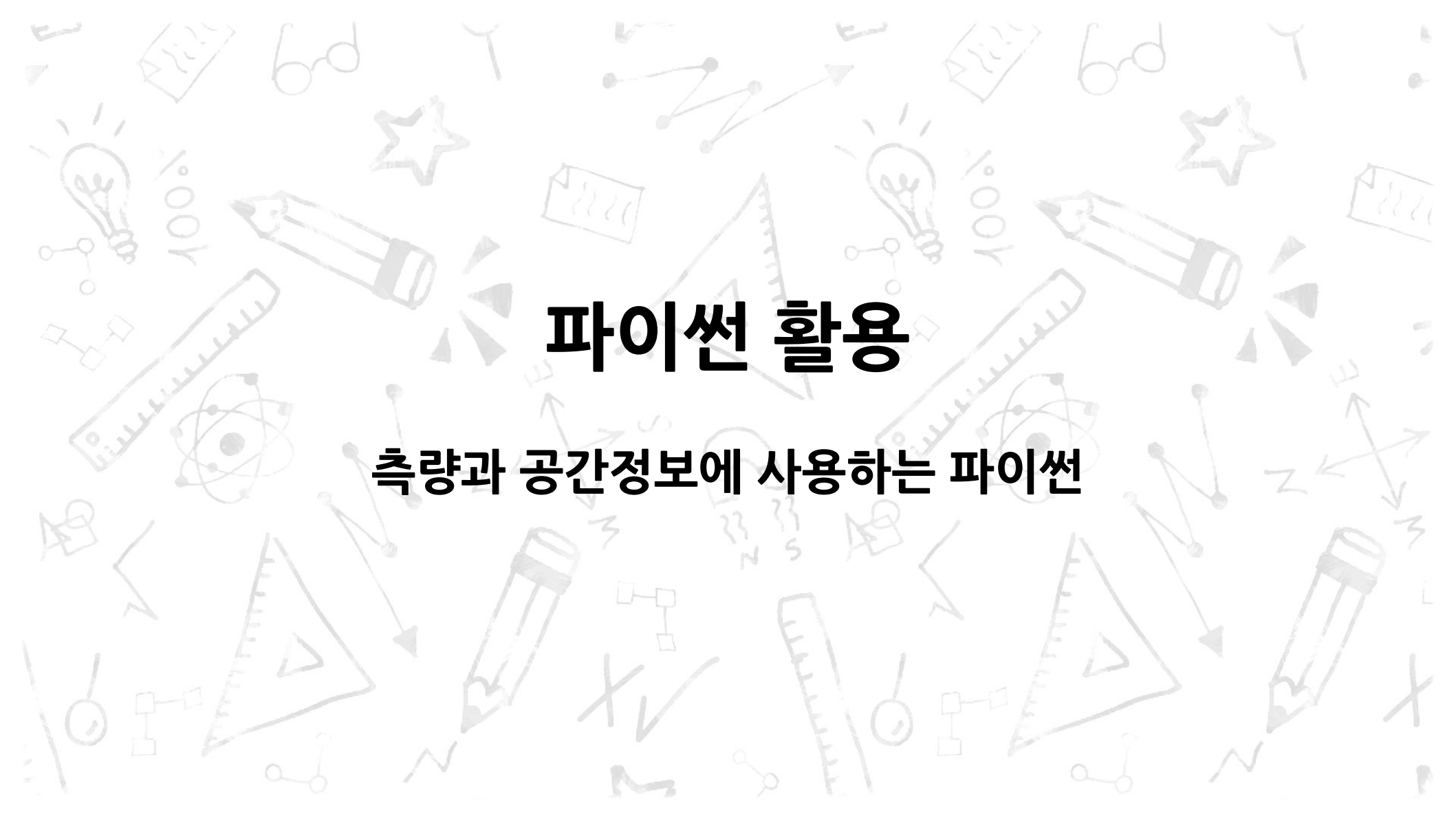


파이썬 활용

AI로 파이썬 프로그램 개발하기

AI와 함께 만드는 파이썬 프로그램

- 전체 목록: <https://profkim2000.github.io/pyjisa/python.html>
 - 숫자맞추기 게임
 - 현재 시각
 - 가위바위보 게임
 - 행맨 게임
 - 행맨 게임 그래픽 버전
 - 오목 게임 그래픽 버전



파이썬 활용

측량과 공간정보에 사용하는 파이썬

측량과 공간정보에 사용하는 파이썬

- 전체 목록: <https://profkim2000.github.io/pyjisa/pygis.html>
 - 좌표 변환
 - 좌표 변환 그래픽 버전
 - 좌표 표시 지도 이미지
 - Shp 파일 자료 뷰어
 - 폴리곤 넓이

참고자료

- python 공식 홈페이지

- <https://www.python.org>

- python 무료 ebook

- 점프 투 파이썬: <https://wikidocs.net/book/1>

- python 고등학교 교과서

- 고등학교 정보(천재교과서)

License

copyright 2026. 김문식 All rights reserved.

이 자료는 국토정보교육원 내에서, 교육 목적의 수업을 위해서만 사용 가능하며,
이 외의 공간에서 저작물을 공유 또는 게시하는 행위는 저작권법 위반에 해당할 수
있습니다.